Experiment : 10

Document Similarity Analysis Using Shingling and Locality Sensitive Hashing (LSH)

Mapped CO(s): CO3

Aim

To identify and evaluate document similarity using Shingling and Locality Sensitive Hashing (LSH) techniques.

Objectives

- To understand the concept of document similarity.
- To generate shingles from text documents.
- To compute document similarity using the Jaccard Similarity measure.
- To understand the concept of Locality Sensitive Hashing (LSH).
- To identify similar documents efficiently using LSH.

Prerequisites

- Basic knowledge of Python programming.
- Basic understanding of sets and hashing.
- Python 3.x installed.
- Jupyter Notebook or Google Colab.
- Required libraries: Pandas, NumPy, Datasketch, and Matplotlib.

Theory/Background

Document similarity is an important task in text mining, information retrieval, plagiarism detection, and search engines. It measures how similar two or more documents are based on their textual content.

Shingling is a technique that divides a document into overlapping sequences of words or characters called *shingles*. Each document is represented as a set of shingles. The similarity between two documents is commonly measured using the **Jaccard Similarity**, which is calculated as the ratio of the intersection of two shingle sets to their union.

Locality Sensitive Hashing (LSH) is an efficient indexing technique used to quickly identify similar documents without comparing every pair directly. It uses MinHash signatures to group similar documents into the same buckets, significantly reducing computation time for large datasets.

These techniques are widely used in duplicate document detection, recommendation systems, and search engines.

Algorithm/Methodology

- Import the required Python libraries.

- Create or load a collection of text documents.
- Generate shingles for each document.
- Calculate Jaccard Similarity between documents.
- Generate MinHash signatures.
- Build an LSH index.
- Query similar documents using LSH.
- Interpret the similarity results.

Dataset Description

Dataset Used: Sample Text Documents

Description:

A small collection of text documents is used to demonstrate document similarity analysis using Shingling and Locality Sensitive Hashing.

Attribute	Description
Document ID	Unique identifier for each document
Text	Document content used for similarity analysis

Program/Implementation

The implementation should include:

- Importing required libraries.
- Creating sample documents.
- Generating shingles.
- Computing Jaccard Similarity.
- Applying MinHash.
- Building an LSH index.
- Querying similar documents.
- Displaying similarity results.

Test Cases

Test Case	Expected Result
Load documents	Documents loaded successfully

Generate shingles	Shingle sets created
Calculate Jaccard Similarity	Similarity score displayed
Build LSH index	Index created successfully
Query similar documents	Similar documents identified

Experimental Results

Observation

- Shingles were successfully generated for all documents.
- Jaccard Similarity scores were computed between document pairs.
- MinHash signatures were created for efficient comparison.
- Locality Sensitive Hashing correctly identified similar documents.
- Similar documents were retrieved quickly without comparing every document pair.

Result

Document similarity analysis was successfully performed using Shingling and Locality Sensitive Hashing (LSH). The experiment demonstrated efficient similarity detection suitable for large-scale text datasets.

Conclusion

The experiment demonstrated the use of Shingling and Locality Sensitive Hashing (LSH) for document similarity analysis. Shingling represented documents as sets of overlapping terms, while LSH efficiently identified similar documents using MinHash signatures. These techniques are widely used in text analytics, plagiarism detection, duplicate document identification, and information retrieval systems.

Exercises/Assignments

1. Compute the Jaccard Similarity between two different documents.
2. Change the shingle size and observe its effect on similarity.
3. Add a new document and identify its similar documents using LSH.
4. Compare the results obtained using different similarity thresholds.
5. Explain the advantages of LSH over direct document comparison.